```
Output:
Hello from Other\\Sub namespace.
Hello from Other\\Sub namespace.
```

Notice the first call does not have a leading backslash, so PHP begins its search in current namespace. But the first call does include a sub-namespace name: `Sub`. Making a qualified call in this way means that PHP will append the sub-namespace name in the call to the current and search there. The current namespace is `Other`, the qualified call asks for the `Sub` sub-namespace, the result is PHP searches `Other\Sub` and calls the function from there.

The second call has a leading backslash, this is a fully qualified name starting from root space, and we get the same version of `collision()` as we can see from the output.

Using Namespaces

Using qualified names is a great way of being explicit about your intentions, but there is another way to achieve the same results and keep your code a little easier to read. Imagine that you were working in one namespace but needed to call a function defined in another. Even worse, the function in question does not collide with any defined where your code happens to be executing. Do you really need to litter your file with lots of calls to `\Some\Deeply\Nested\Sub\Sub\Space\usefulFunction()`? The good news is that you don't, because you can import symbols (functions, interfaces, classes, and constants) from other namespaces with the keyword `use`. To use a function from a specific namespace without qualifying on every call, add the statement `use function`.

```
namespace Some\Other\Space {
    use function \Other\collision;

    collision();
}
```

```
Output:
Hello from Other namespace.
```

Every call to `collision()` after PHP evaluates the `use function` statement will be made to `\Other\collision()`. If for some reason you need to use identically named functions from multiple namespaces, it is possible to create an *alias* using the keyword `as`.